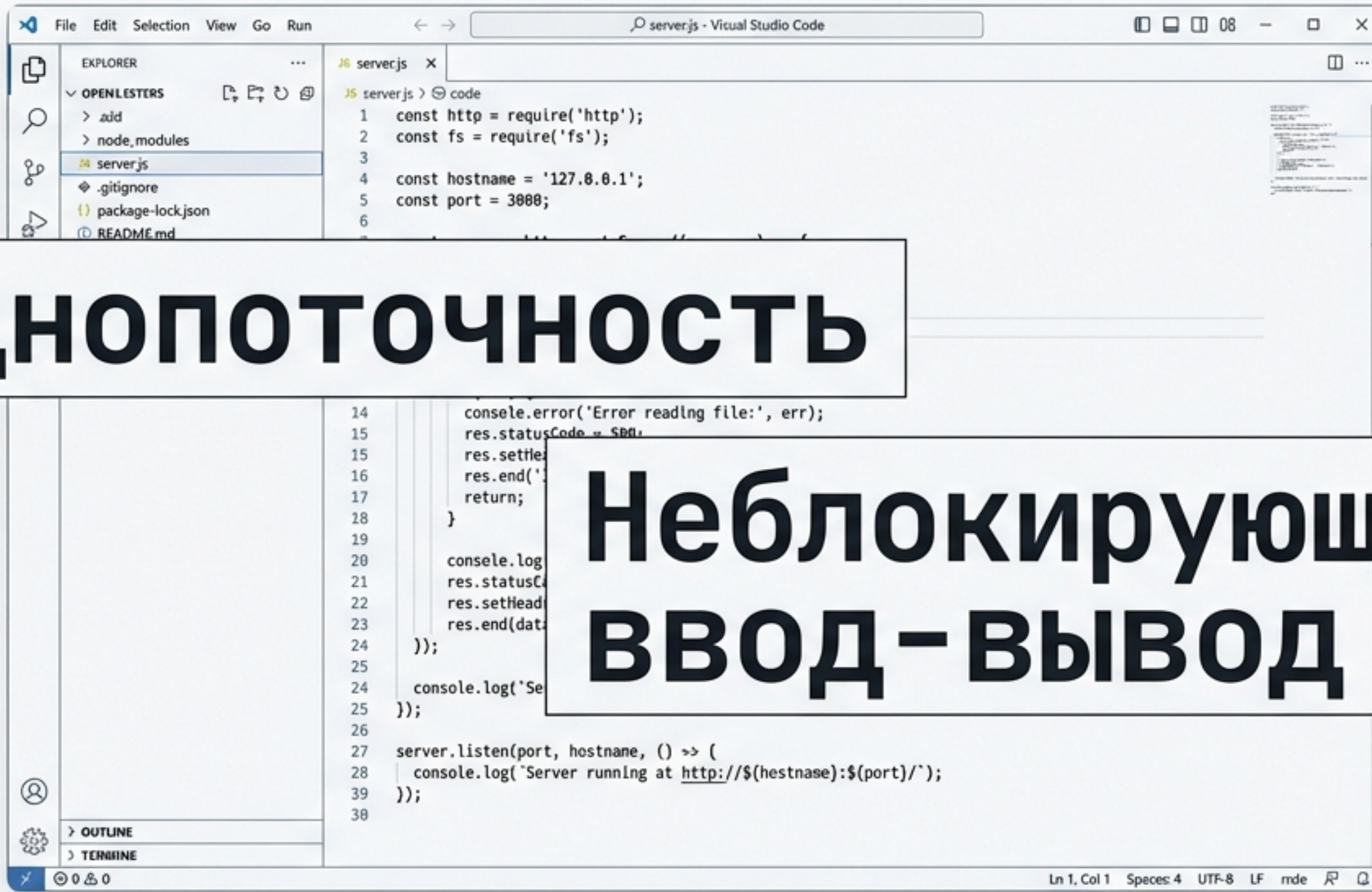




Однопоточность

Неблокирующий ВВОД-ВЫВОД

Как Node.js совмещает, казалось бы, несовместимое?
Сценарий «Раскрытие слоя».

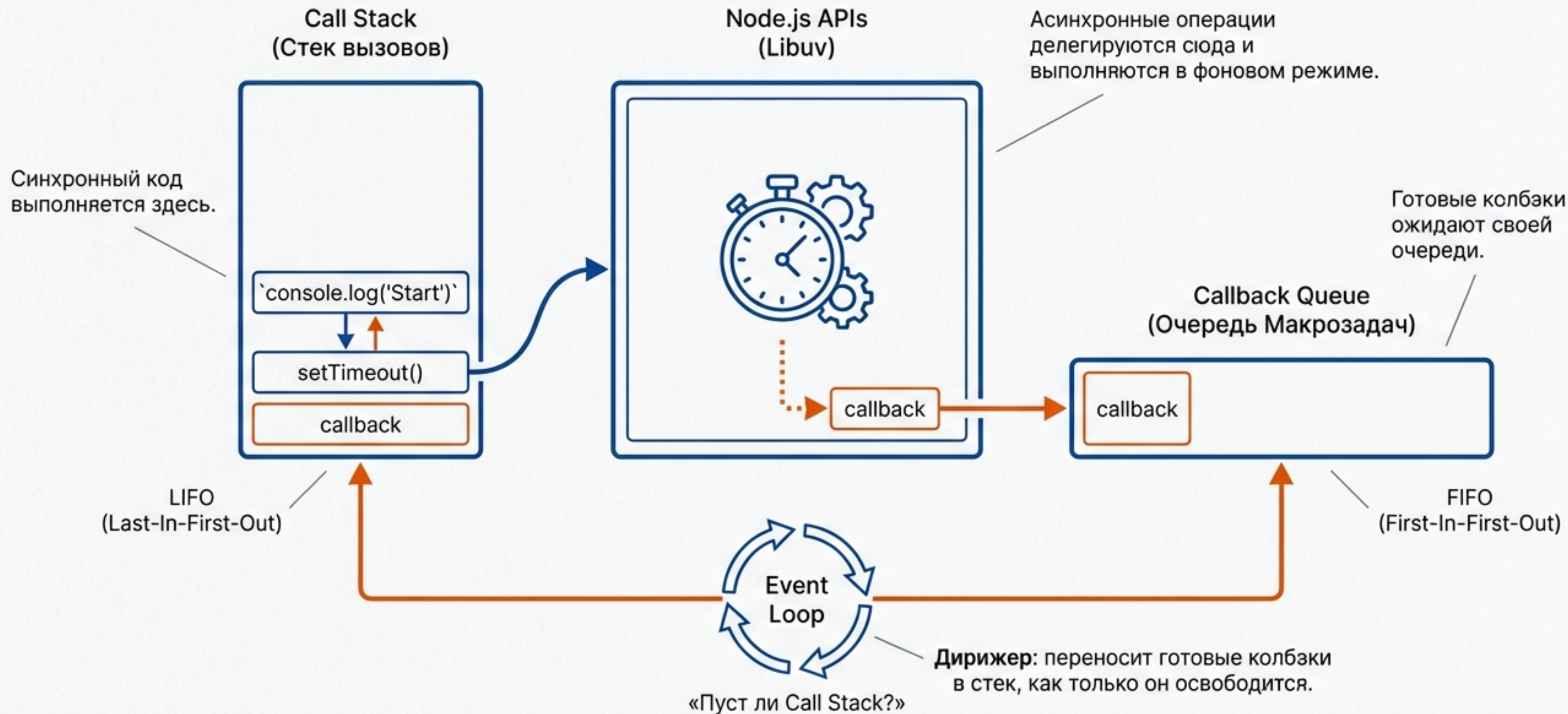
Архитектура Node.js: Крупный план.



JavaScript выполняется в одном потоке (V8), но тяжелые операции ввода-вывода делегируются системным механизмам через Libuv.

Загрузка концепции: 25%

Танец компонентов: Call Stack, API и Очереди.



Не все задачи созданы равными: Макрозадачи vs. Микрозадачи.

Очередь Макрозадач (Macrotask Queue)

Более крупные, отложенные операции.



- `setTimeout`, `setInterval`



- `setImmediate`



- Операции ввода-вывода (I/O)
`fs.readFile`, сетевые запросы

Очередь Микрозадач (Microtask Queue)

Короткие, высокоприоритетные задачи.



- Колбэки промисов
`.then()`, `.catch()`, `.finally()`

- `async/await` (синтаксический сахар над промисами)



- `queueMicrotask()`

Event Loop отдает предпочтение микрозадам.
Они выполняются между макрозадачами.

Проверка интуиции: что выведется в консоль?

```
console.log('1. Синхронный код');  
  
setTimeout(() => console.log('2. Таймер (макрозадача)'), 0);  
  
Promise.resolve().then(() => console.log('3. Промис (микрозадача)'));  
  
process.nextTick(() => console.log('4. nextTick'));  
  
console.log('5. Синхронный код');
```

В каком порядке появятся сообщения в консоли?
Подумайте секунду, прежде чем перевернуть страницу.

Иерархия приоритетов: Правила игры Event Loop.

1. Синхронный код

Выполняется немедленно, пока не опустеет Call Stack.

2. Очередь `process.nextTick()`

Выполняется сразу после текущей операции, до перехода Event Loop к следующей фазе. Это не микрозадача, а отдельная, самая приоритетная очередь.

3. Очередь Микрозадач

Выполняется *полностью* после ``nextTick`` и перед следующей макрозадачей.

4. Очередь Макрозадач

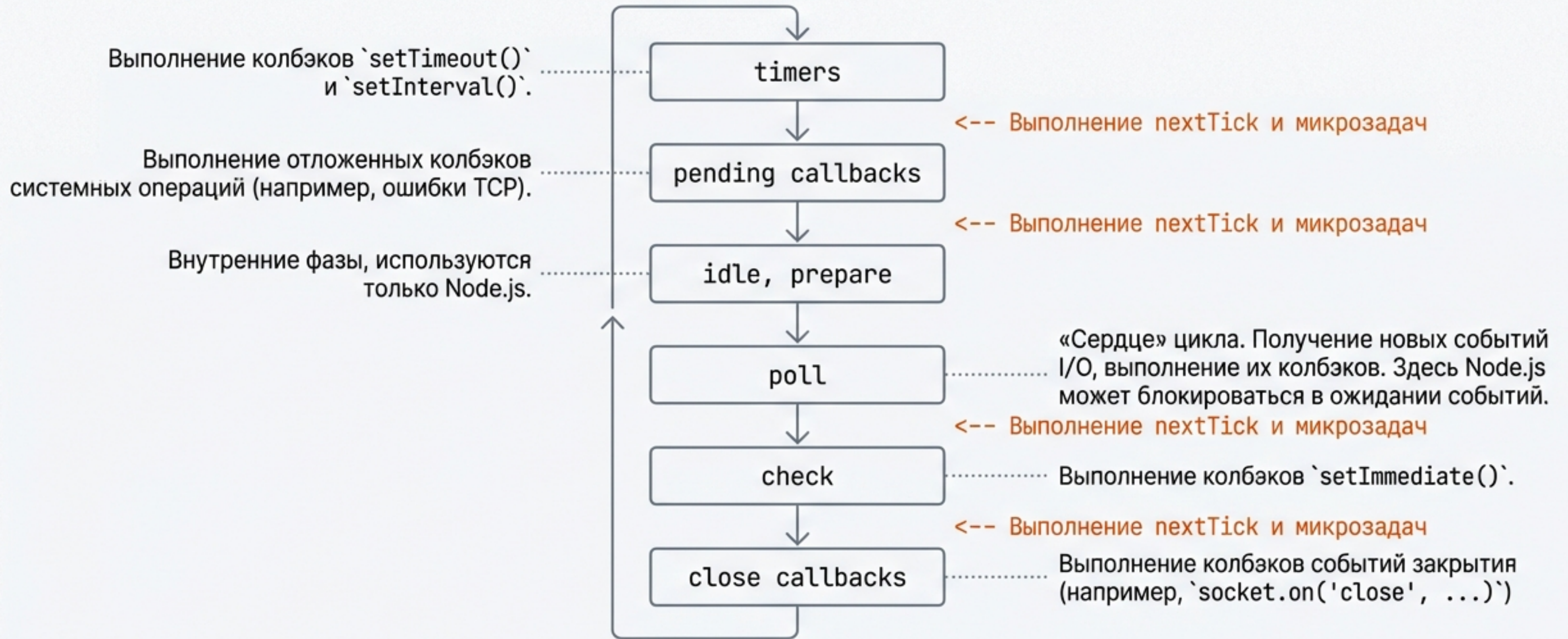
Выполняется *одна* задача из очереди соответствующей фазы.

>_ Console Output

1. Синхронный код
5. Синхронный код
4. `nextTick`
3. Промис (микрозадача)
2. Таймер (макрозадача)

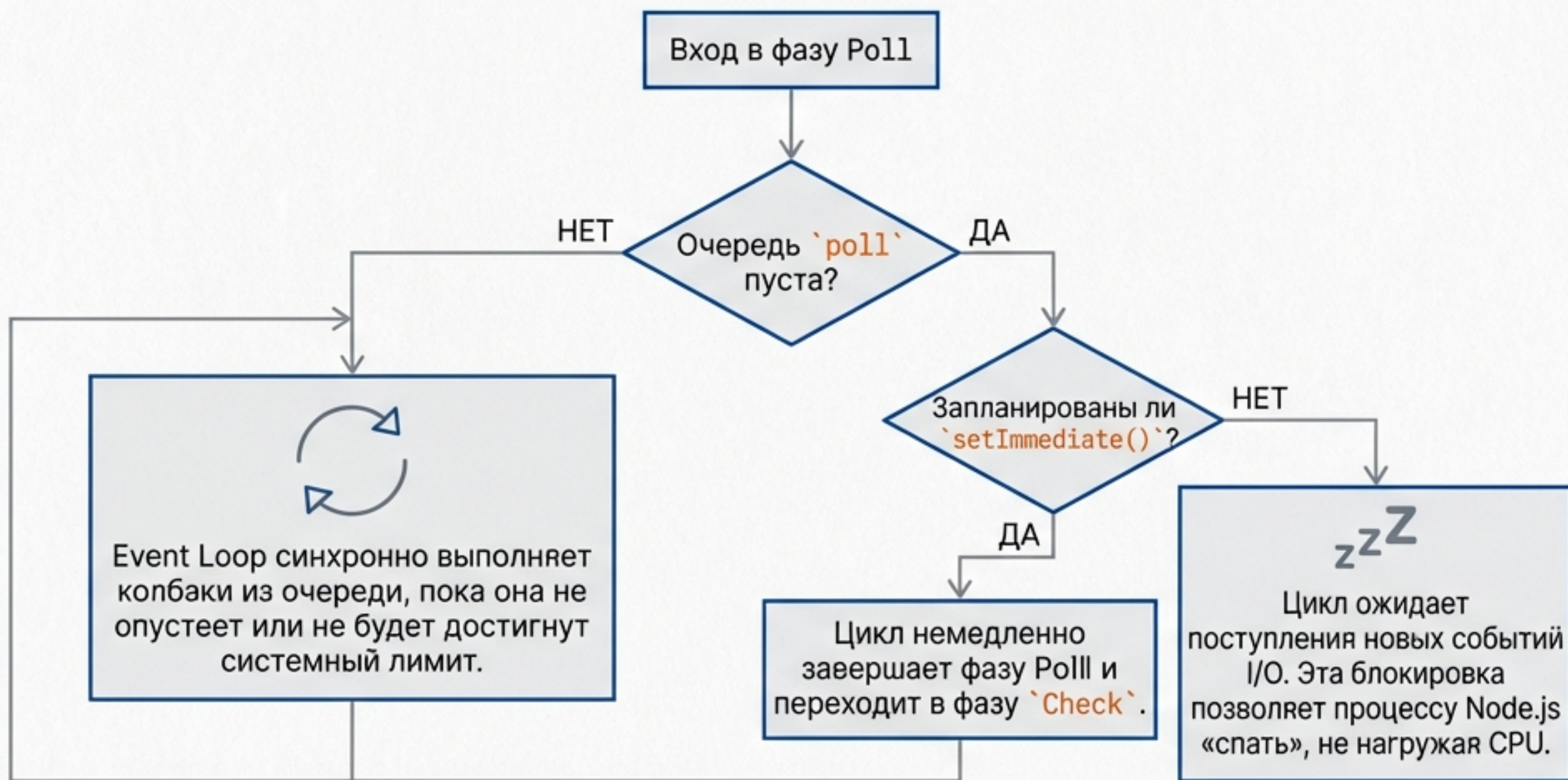
Вывод: После каждой выполненной макрозадачи Event Loop *сначала* полностью опустошает очередь ``nextTick``, а затем очередь микрозадач, и только потом переходит к следующей макрозадаче.

Анатомия цикла: Шесть фаз Event Loop в Libuv.



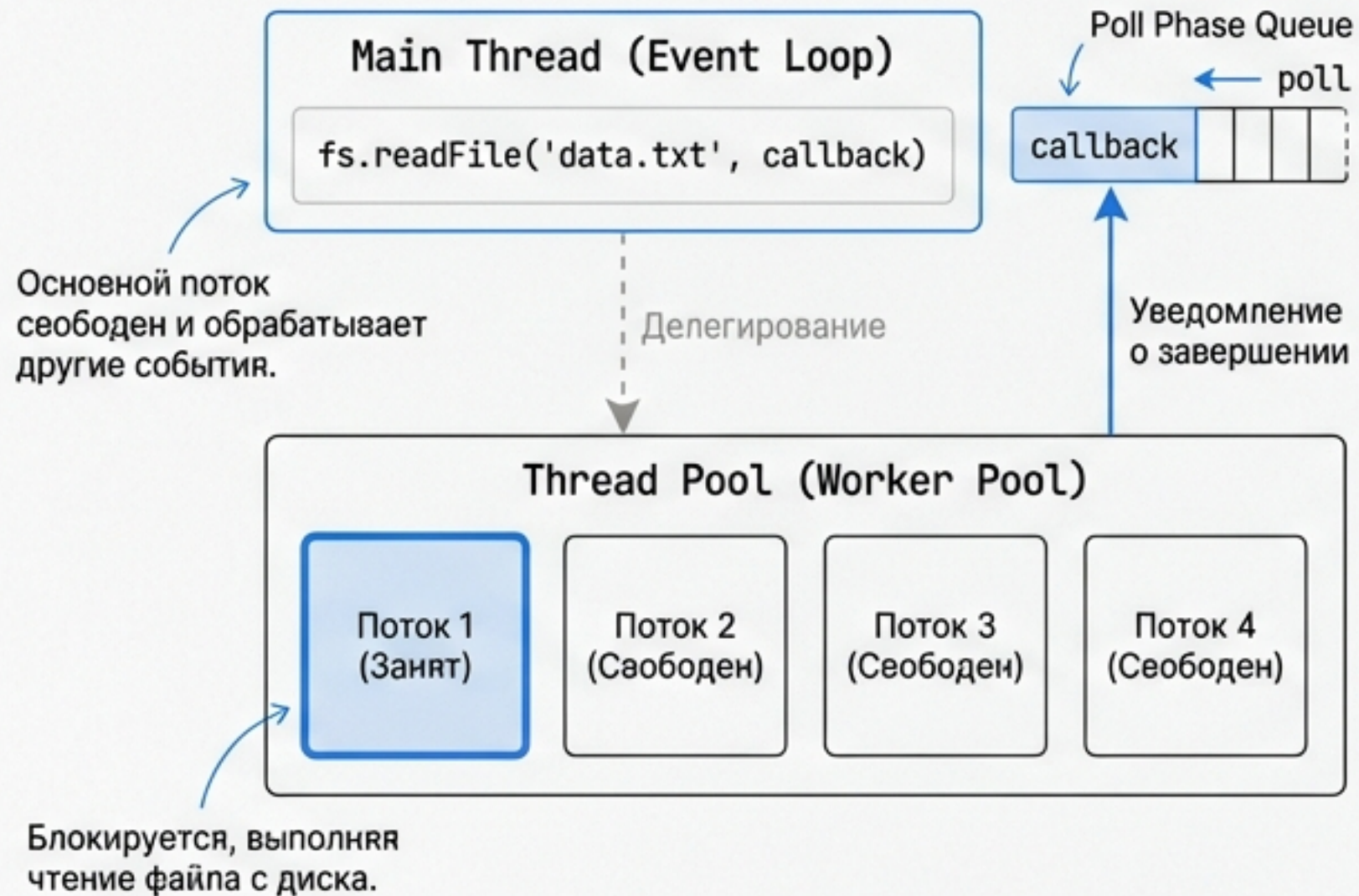
Примечание: Очереди ``nextTick`` и микрозадач выполняются между каждой из этих фаз.

Фаза Poll: Работа и Ожидание



Фаза Poll — это место, где Node.js эффективно использует системные механизмы уведомлений (epoll, kqueue, IOCP) для обработки тысяч соединений без создания тысяч потоков. Это и есть тот самый «событийный демultipлексор».

Раскрываем миф: что на самом деле делает тяжелую работу?



Задачи, выполняемые в Thread Pool:

- Все операции модуля `'fs'` (кроме синхронных).
- Некоторые операции модуля `'crypto'` (например, `'crypto.pbkdf2'`).
- Операции DNS (`dns.lookup()`).
- Некоторые операции сжатия (`'zlib'`).

Node.js **однопоточен** только для вашего JavaScript кода. **Libuv** использует **МНОГОПОТОЧНОСТЬ** для асинхронной эмуляции блокирующих системных вызовов.

Загрузка концепции: 75%

Пул потоков в действии: Ограничение по умолчанию.

```
const crypto = require('crypto');
const start = Date.now();

// Запускаем 5 "тяжелых" операций
for (let i = 0; i < 5; i++) {
  crypto.pbkdf2('password', 'salt', 100000, 512, 'sha512', () => {
    console.log(`Hash ${i + 1}:`, Date.now() - start, 'ms');
  });
}
```

```
Hash 4: 850 ms
Hash 2: 855 ms
Hash 1: 860 ms
Hash 1: 860 ms
Hash 3: 862 ms
Hash 5: 1690 ms <- Заметная задержка
```

Первые четыре вызова занимают четыре потока в пуле и выполняются параллельно. Пятый вызов ждет, пока один из потоков освободится, поэтому его время выполнения примерно в два раза больше.

Как управлять:

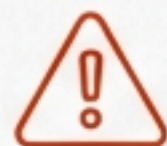
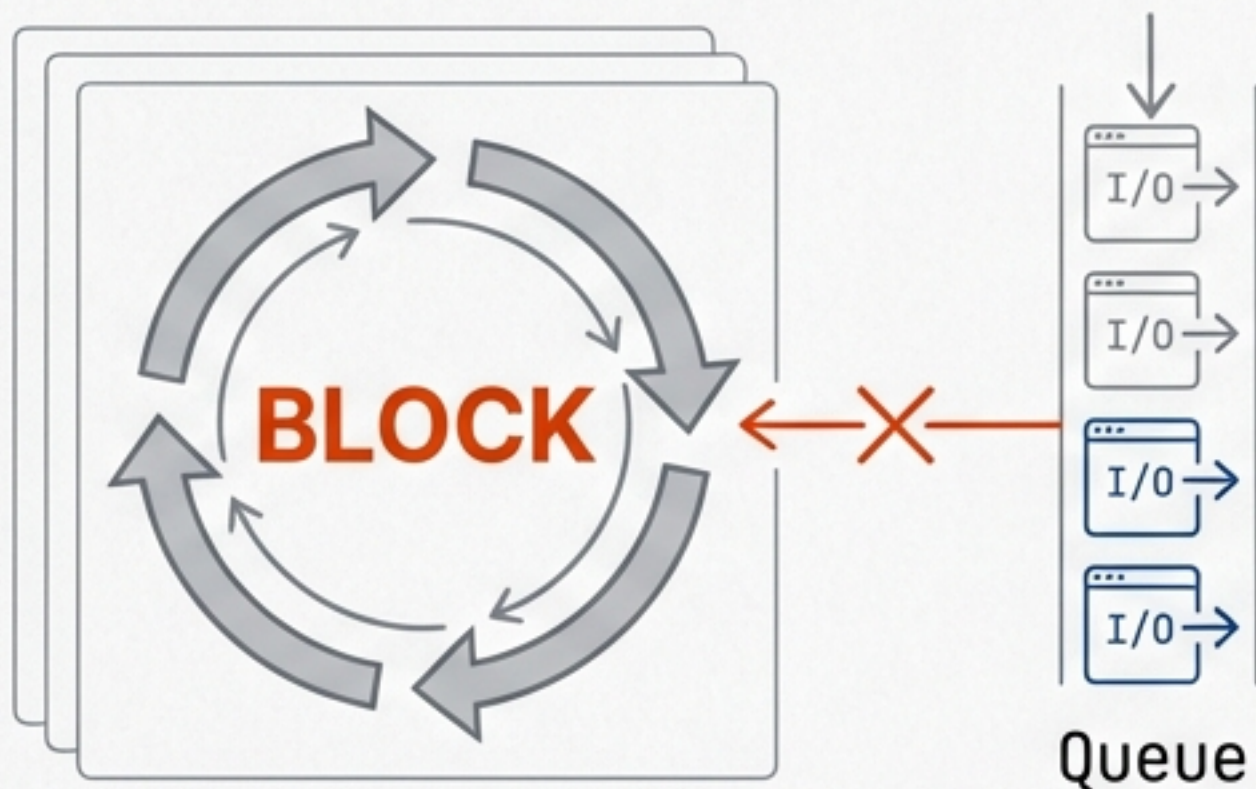
Размер пула можно изменить переменной окружения `UV_THREADPOOL_SIZE`.

Главное правило: Не блокируй Event Loop!

Блокирующий синхронный код



```
const data = fs.readFileSync('/path/to/file');
```

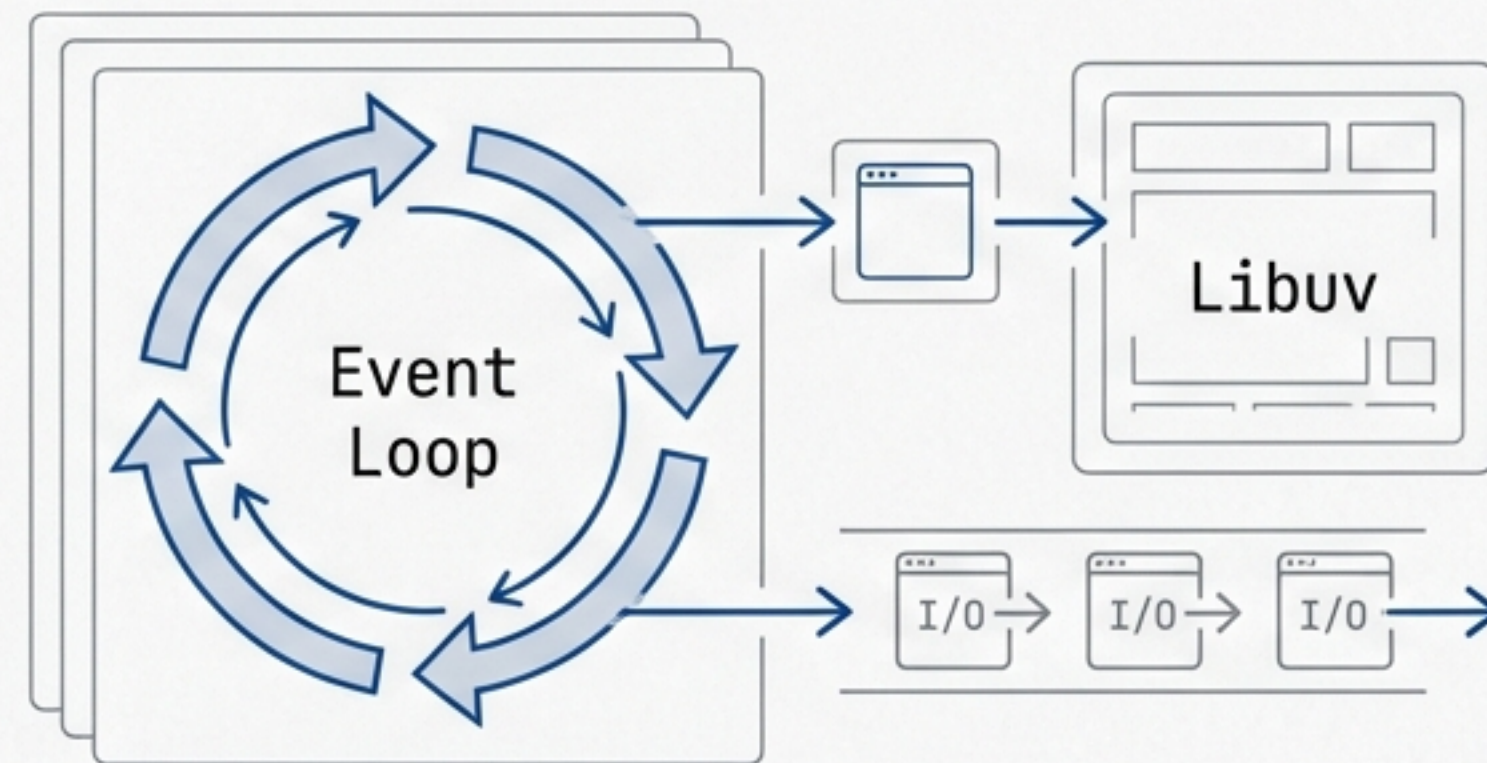


- Сервер перестает отвечать.
- Запросы таймаутятся.
- Производительность падает до нуля.

Неблокирующий асинхронный код



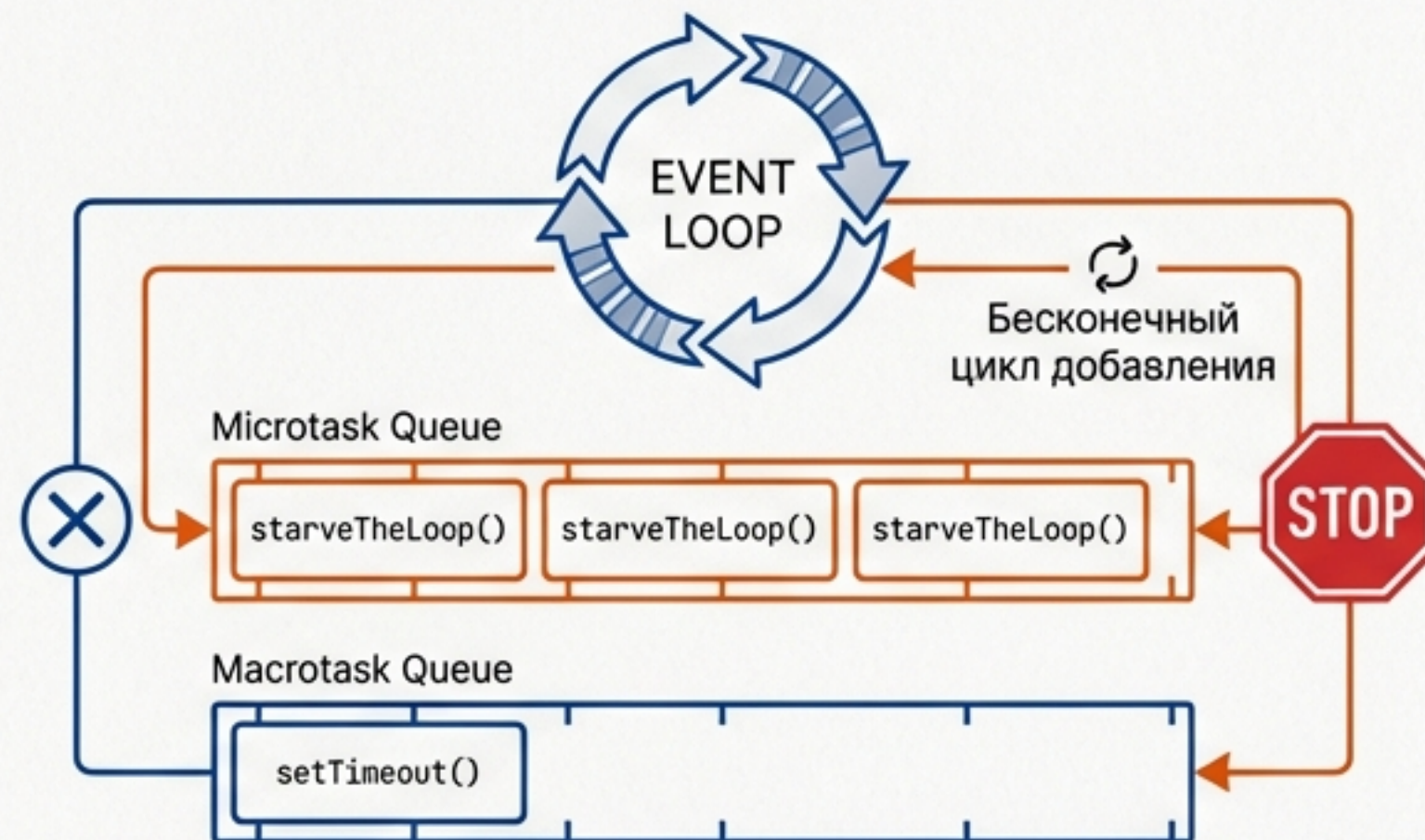
```
fs.readFile('/path/to/file', (err, data) => { ... });
```



- Высокая пропускная способность.
- Отзывчивость приложения.

Опасность микрзадач: «Голодание» Event Loop.

```
setTimeout(()=> console.log('Таймер (макрозадача)'), 0);  
  
// Эта функция создаст бесконечный цикл микрзадач  
function starveTheLoop() {  
  Promise.resolve().then(starveTheLoop);  
}  
  
starveTheLoop();
```



Функция `starveTheLoop` постоянно добавляет новые микрзадачи в очередь. Поскольку очередь микрзадач должна быть полностью опустошена перед переходом к следующей макрозадаче, Event Loop никогда не доберется до фазы `Timers`, чтобы выполнить `setTimeout`.

Рекурсивные вызовы `process.nextTick()` или промисов без выхода из цикла — верный способ заблокировать обработку I/O и таймеров.

Стратегии оптимизации: Как сохранить Event Loop здоровым.

Подход	Описание	Рекомендуемый сценарий
Разбиение задач (Task Partitioning)	Длинная синхронная операция (например, обработка большого массива) разбивается на части. Между частями управление возвращается Event Loop через <code>setImmediate()</code> .	Обработка больших объемов данных без блокировки.
Worker Threads	Ресурсоемкие вычисления (CPU-bound) выносятся в отдельный поток со своим Event Loop и V8. Обмен данными через <code>postMessage()</code> или <code>SharedArrayBuffer</code> .	Обработка изображений, криптография, сложные математические расчеты.
Child Processes / Cluster	Запуск полностью изолированных процессов Node.js. Используется для масштабирования на несколько ядер CPU и повышения отказоустойчивости.	Создание веб-серверов, способных утилизировать все ядра процессора.

Управление жизненным циклом: `ref()` и `unref()`.

Теория

- Процесс Node.js завершается, когда в Event Loop не остается активных «хэндлов» (handles).

Что такое хэндл?

Долгоживущий объект, представляющий ресурс I/O (таймер, сокет, сервер).

- `timer.ref()` (поведение по умолчанию):



«Держит» процесс активным. Пока этот таймер существует, приложение не завершится.

- `timer.unref()`:



Позволяет таймеру или сокету работать, но не мешает процессу завершиться. Event Loop не будет ждать этот хэндл.

```
const interval = setInterval(() => {
  console.log('Этот интервал не даст процессу
  завершиться.');
```



```
}, 1000);

// Долгоемоечение трзнов - ключевой вызов
const unrefedInterval = setInterval(() => {
  console.log('А этот интервал не будет мешать выходу.');
```

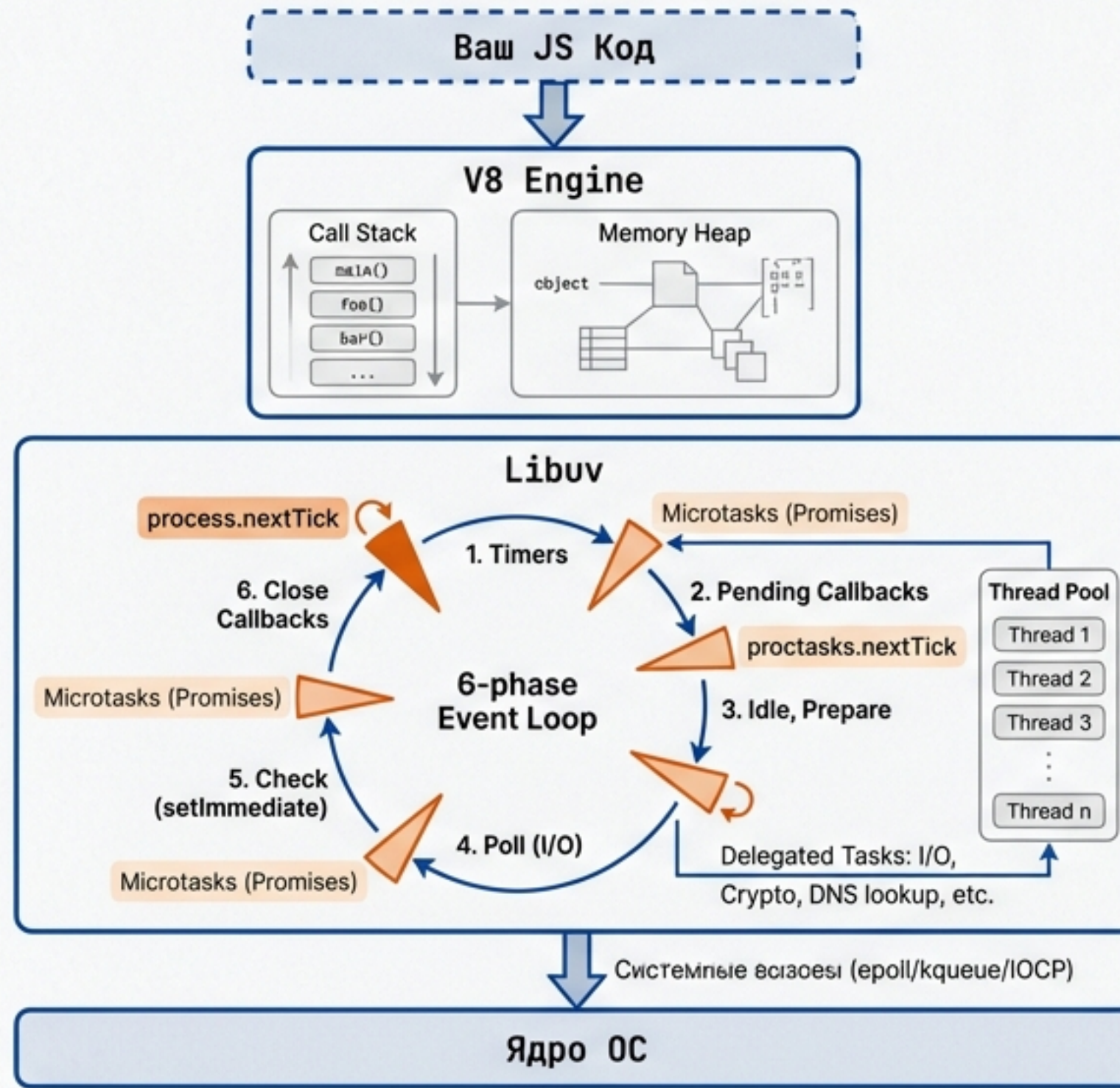


```
}, 1000);
unrefedInterval.unref();
```

Ключевой вызов, разрешающий завершение

Применение: Фоновые задачи, мониторинг, периодическая очистка кэша, которые не должны быть основной причиной работы приложения.

Полная картина: От кода до ядра системы.



Ключевые выводы

1. **Сердце Node.js — это Event Loop**, управляемый Libuv.
2. **Иерархия — это всё:**
Синхронный код > `nextTick` > Микрозадачи > Макрозадачи.
3. **Не блокируйте основной поток:** Выносите тяжелые вычисления в `Worker Threads`.
4. **«Однопоточность» — это миф:** Libuv использует потоки для эмуляции асинхронности.